User Flows (Concise)

Version: 2025-08-23

Architecture Snapshot

Client (index.html + extras.js + app.js) ↔ API (Express /api/*) ↔ DB Service Worker: cache shell + network-first HTML, cache-first assets. State: localStorage token (ft.jwt) + offline POST outbox.

Core Flows

1. Register

- Action: Click Register.
- POST /api/auth/register { email, password } → token saved → bootApp.
- Error: 400 duplicate → alert.

2. Login

- POST /api/auth/login → token saved → bootApp.
- Error: 401 invalid credentials → alert.

3. Boot (Authenticated Session)

- Hide auth, show app, fetch /api/me (if needed), build stations, flush outbox, load today (/api/entries/:date), set progress.
- Failure: clear token, return to auth.

4. Update Sets

- - / - buttons adjust 0..MAX_SETS; progress recalculated locally.

5. Save Progress

- Collect counts + weight.
- POST /api/entries (or queue if offline). Debounce identical within 2s.
- Toast: saved | queued | failed.

6. History

- GET /api/entries → de-dup by date → editable modal (inline save/delete).
- Save: POST /api/entries (upsert). Delete: DELETE /api/entries/:date.

7. Summary

- GET /api/summary → weekly rows (weight avg + totals). Empty state message if none.

8. Logout

- Clear ft.jwt → reload (auth screen).

9. Offline Queue

- Any POST while offline: enqueue (replace same-date entry). Flush on online/boot.

10. Dev Bypass

- ?dev=1 / #dev or localStorage flag → mock api() responses → boot without auth.

11. Face ID Demo

- Icon click → simulated scan → attempt login with entered or demo creds.

Error Handling (Minimal)

- 401: token cleared → auth screen.
- Offline POST: queued with toast.
- Save duplicate (<2s): ignored.
- History filter removes invalid (legacy) rows.

Data Shapes

Entry POST: { date:'YYYY-MM-DD', weight, fitnessData:{ ExerciseName: sets } } Entry GET: { entry_date, weight, fitness_data } Summary Row: { week_of_sunday, weight_avg, totals:{} }

Key Files

- web/app.js: core logic (auth, stations, save, history, summary, offline)
- web/extras.js: diagnostics, UI helpers, fetch logging
- web/service-worker.js: caching strategy
- server/server.js: Express routes + JWT guard

Risks / Next Steps (Top 5)

1. Space-containing DOM IDs (exercise names) → slugify.
2. Cache-first JS → potential staleness → adopt stale-while-revalidate or hashed builds.
3. localStorage fallback volatility → warn & consider IndexedDB.
4. UTC date normalization (avoid timezone drift).
5. Accessibility: add dialog roles & focus management.

Quick QA Matrix

Flow	Verify
Register/Login	Token stored, app boots
Save Offline	Toast queued; sync after reconnect
History Edit/Delete	Persists and refreshes list
Summary Empty	Shows placeholder row
Dev Bypass	App loads without network calls

End.